

PopOut: MCTS & ID3 DT

Ana Francisca Araújo,
Pedro N. Jorge,
Sofia Ribeiro

University of Porto

May 17, 2026

Outline

Part 1: Game Development

Part 2: Monte Carlo Tree Search

Part 3: ID3 Decision Tree

Future Work

Outline

Part 1: Game Development

Part 2: Monte Carlo Tree Search

Part 3: ID3 Decision Tree

Future Work

PopOut Implementation

- ▶ Two bitboards (*self.player* and *self.opponent*) in order to represent a state. Each bitboard is of the format:

[col/6]0[col/5]0[col/4]0[col/3]0[col/2]0[col/1]0[col/0]

- ▶ *self.cur_player* to different between players (0: Player 1; 1: Player 2).
- ▶ Zobrist hashing (used in Chess engines) to handle three-fold repetitions using *self.player*, *self.opponent* and *self.cur_player*.

```
      *
- - - - -
- - - - -
- 0 - 0 - - -
x x x 0 - - -
0 x 0 x x - -
0 x 0 x 0 - -

Player X's turn
←/→ move | d drop | p popout | t draw | q quit
```

Command Line Interface

- ▶ Developed using the Python *curses* library to create an interactive terminal interface responsible for:
 - ▶ Drawing the board
 - ▶ Displaying menus
 - ▶ Handling keyboard input
 - ▶ Showing game status/messages
- ▶ Supports different game modes:
 - ▶ Player vs. Player
 - ▶ Player vs. MCTS AI
 - ▶ MCTS AI vs ID3 AI
- ▶ Board rendering:
 - ▶ $\leftrightarrow$ - move cursor
 - ▶ d - drop piece
 - ▶ p - popout piece
 - ▶ t - declare draw
 - ▶ q - quit game

Command Line Interface

- ▶ Features:
 - ▶ Dynamic colored interface using terminal color pairs
 - ▶ ASCII-art title screen and interactive menu system
 - ▶ Real-time board updates after every move
 - ▶ Draw detection:
 - ▶ Full board
 - ▶ Three-fold repetition
 - ▶ Logging system (*debug.log*) used for debugging AI/game states
- ▶ Game loop:
 - ▶ Main loop continuously:
 - ▶ Draws board
 - ▶ Gets player/AI moves
 - ▶ Validates move
 - ▶ Updates game state
 - ▶ Checks victory/draw conditions
 - ▶ Turn switching handled automatically

Outline

Part 1: Game Development

Part 2: Monte Carlo Tree Search

Part 3: ID3 Decision Tree

Future Work

MCTS Implementation

- ▶ Implemented a non-standard MCTS by starting with the classic implementation and modifying it given inefficiencies and problems that would emerge

Problem 1: Memory and Statistics

- ▶ **Problem:** In classic MCTS, the same state can be represented with different nodes that don't share the same statistics.
- ▶ **Solution:** Create a Node Transposition Table and a node cache.

Problem 2: Symmetry

- ▶ **Problem:** Can't handle the inherent symmetries of the game states.
- ▶ **Solution:** Create equivalence classes for a state and its mirror, and choose as a representative the one with the smallest hash

$$\text{repr}([s]) = \phi(s) = \min(\text{Zobrist}(s), \text{Zobrist}(\text{mirror}(s))).$$

Problem 3: UCB1 Formula Assumption

- **Problem:** The assumption of the UCB1 formula is violated

$$N(n) \leq N(\text{PARENT}(n))$$

- **Solution:** Extend the UCB definition using edge statistics, where an edge is given by (parent state, action). Assumption changes:

$$N(s, a) \leq \sum_{k \in \mathcal{A}(s)} N(s, a).$$

The UCB1 formula changes:

$$\text{UCB1}(n) = Q(s, a) + C \cdot \sqrt{\frac{\log \sum_{k \in \mathcal{A}(s)} N(s, a)}{N(s, a)}},$$

where $Q(s, a) = W(s, a)/N(s, a)$.

Problem 4: Duplicate Edges

- **Problem:** Duplicate edges due to creating state equivalence classes. We can now have

$$(s, a), (s, a') \mapsto s'$$

- **Solution:** Create equivalence relation/classes for valid actions in a state s :

$$a \sim_{\mathcal{A}(s)} a' \iff \phi(\tau(s, a)) = \phi(\tau(s, a')).$$

Note: we are using s instead of $\text{repr}([s])$ for notation simplicity.

Problem 5: Trouble Findings Obvious Moves

- ▶ **Problem:** Can't correctly identify obvious best move: immediate win or block opponent's win, due to random rollout policy and branching factor $b \simeq 8$.
- ▶ **Solution:** Use an heuristic that check if we can win or block opponent's win, and if not choose a random move.

Outline

Part 1: Game Development

Part 2: Monte Carlo Tree Search

Part 3: ID3 Decision Tree

Future Work

ID3 DT Implementation

- ▶ The implemented algorithm recursively builds the decision tree by continuously splitting the dataset on the attribute with the highest information gain.

$$H(C) = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$$
$$Gain(A) = I(C; A) = H(C) - H(C | A)$$

- ▶ It obtained 90% test accuracy on the iris dataset.

Dataset Generation

- ▶ Generated a dataset with MCTS self-play with 800 simulations and $C = \sqrt{2}$. We used the following format per row: (state, PMF, $a \sim \text{PMF}$)
- ▶ The chosen action for the state was selected from the PMF *self.search* for sake of diversity
- ▶ We aggregated all entries of the same state on the action with more counts

ID3 on MCTS dataset

- ▶ To measure ID3 DT performance we used the following metrics:
 - ▶ Valid Move %: 77.8%
 - ▶ Top-2 Move Accuracy: 14%
- ▶ It overfitted, memorizing the data rather than learning from it (expected).
- ▶ Since it was not perfect on predicting legal moves, we made some constraints in the AI vs AI game.

Outline

Part 1: Game Development

Part 2: Monte Carlo Tree Search

Part 3: ID3 Decision Tree

Future Work

CNN Rollout Policy

- ▶ Currently, we are still making a dataset that is big enough to be used for training a CNN.
- ▶ This CNN will be swapped with the current heuristic rollout in order to get a more accurate PMF per state in less simulations.